Data source excerpt (continued) *data_source.py*

```
RAW_ASCII_DIR.mkdir(parents=True, exist_ok=True)
ICAO_EXCEL_DIR.mkdir(parents=True, exist_ok=True)

# ---------------------------------------------------------------------------
# Effective settings  (config file ? env var ? hard-coded default)
# ---------------------------------------------------------------------------
SIMULATION_MODE = _cfg_get(_cfg, "mode", "simulation_mode",
                           "SIMULATION_MODE", "true").strip().lower() == "true"
SIMULATION_REPLAY_BATCH = int(_cfg_get(_cfg, "simulation", "replay_batch",
                                       "SIMULATION_REPLAY_BATCH", "20"))
SIMULATION_REPLAY_LOOP = _cfg_get(_cfg, "simulation", "replay_loop",
                                  "SIMULATION_REPLAY_LOOP", "false").strip().lower() == "true"
SIMULATION_STALE_SECONDS = int(_cfg_get(_cfg, "simulation", "stale_seconds",
                                        "SIMULATION_STALE_SECONDS", "12"))

PINGSTATION_LISTEN_HOST = _cfg_get(_cfg, "live", "listen_host",
                                   "PINGSTATION_LISTEN_HOST", "0.0.0.0")
PINGSTATION_LISTEN_PORT = int(_cfg_get(_cfg, "live", "listen_port",
                                       "PINGSTATION_LISTEN_PORT", "30003"))
PINGSTATION_STALE_SECONDS = int(_cfg_get(_cfg, "live", "stale_seconds",
                                         "PINGSTATION_STALE_SECONDS", "45"))
PINGSTATION_CAPTURE_ENABLED = _cfg_get(_cfg, "live", "capture_enabled",
                                       "PINGSTATION_CAPTURE_ENABLED", "true").strip().lower() == "true"
PINGSTATION_CAPTURE_FILE = os.getenv(
    "PINGSTATION_CAPTURE_FILE",
    str(SESSION_DIR),
)

print(
    f"[Config] Mode: {'SIMULATION' if SIMULATION_MODE else 'LIVE'}"
    + (f" | Listening on port {PINGSTATION_LISTEN_PORT}" if not SIMULATION_MODE else "")
    + f" | Config: {_CFG_PATH}"
)


# Single shared decoder instance/algorithm for both simulation replay and the
# live PingStation feed ? simulation exists only to verify this decoder works
# correctly, so it must run through the exact same code path.
_decoder = PingstationRawDecoder()
_receiver = None
_capture_logger = (
    PingstationRawCaptureLogger(PINGSTATION_CAPTURE_FILE)
    if PINGSTATION_CAPTURE_ENABLED or SIMULATION_MODE
    else None
)
_replay_entries = []
_replay_index = 0
_replay_file_mtime = None
_replay_file_path = None
_mode_lock = threading.RLock()


def _ensure_capture_logger():
    global _capture_logger
    if _capture_logger is None:
        _capture_logger = PingstationRawCaptureLogger(PINGSTATION_CAPTURE_FILE)
    return _capture_logger


def set_simulation_mode(enabled):
    """Switch data sources immediately and persist the choice beside the app/exe."""
    global SIMULATION_MODE, _receiver, _replay_index

    enabled = bool(enabled)
    with _mode_lock:
        if _receiver is not None:
            _receiver.stop()
            _receiver = None

        SIMULATION_MODE = enabled
        _decoder.reset_state()
        _replay_index = 0
        if enabled or PINGSTATION_CAPTURE_ENABLED:
            _ensure_capture_logger()

        if not _cfg.has_section("mode"):
            _cfg.add_section("mode")
        _cfg.set("mode", "simulation_mode", "true" if enabled else "false")
        try:
            config_text = _CFG_PATH.read_text(encoding="utf-8")
            replacement = f"simulation_mode = {'true' if enabled else 'false'}"
            updated_text, replacements = re.subn(
                r"(?im)^\s*simulation_mode\s*=\s*.*$",
                replacement,
                config_text,
                count=1,
            )
            if replacements == 0:
                updated_text = config_text.rstrip() + f"\n\n[mode]\n{replacement}\n"
            _CFG_PATH.write_text(updated_text, encoding="utf-8")
        except OSError as exc:
            print(f"[Config] Could not persist simulation mode: {exc}")

    return get_status()


def _get_simulation_log_file():
    return os.getenv("SIMULATION_SOURCE_FILE", DEFAULT_SIMULATION_LOG_FILE).strip()


def _is_useful_record(record):
    lat = float(record.get("latDD", 0) or 0)
    lon = float(record.get("lonDD", 0) or 0)
    return not (lat == 0 and lon == 0)
```